Fine-grained Emotion Classification on Text

1 Problem Description

The final goal of this project is to model how emotions evolve over the course of a Reddit post/thread by applying a sentence-level Hidden Markov Model (HMM). Rather than analyzing each comment in isolation, we assume each comment expresses a single emotion and focus on modeling how these emotions change over a Reddit post/thread.

It is important for researchers and platform designers to understand how emotions evolve within these threads to better monitor, interpret, and manage the emotional tone of online communities.

It is suited to probabilistic reasoning and learning since by using HMM, we treats emotions as latent random states underlies each point in the discussion and evolves as the thread progresses, with each sentence (or comment) generated by the emotional state at that step.

2 Data Sourcing and Processing

2.1 Raw GoEmotions Dataset

We use the GoEmotions dataset introduced by Demszky et al. (2020) and released by Google Research. GoEmotions is a human-annotated corpus of approximately 58k English Reddit comments collected from popular subreddits, where each comment is labeled with zero or more of 28 fine-grained emotion categories plus a `neutral` tag.

In our implementation, we work with the three CSV files provided in the official release. Each row corresponds to one annotation of a Reddit comment and contains:

- Metadata: `id` (comment ID), `text`, `subreddit`, `link_id` (post ID), `parent_id` (parent comment/post), `created_utc`, etc.
- A binary indicator for each of the 28 emotion categories (`admiration`, `amusement`, ..., `neutral`).

A single comment may appear multiple times if it was annotated by multiple raters.

2.2 Aggregating Raters and Constructing Emotion Labels

Since our HMM operates at the comment level, we first aggregate multiple annotations for the same comment ID. For each comment, we sum the binary labels over all raters to obtain an integer count for each of the 28 emotions. We then define a single "main" emotion per comment as the category with the highest count (breaking ties arbitrarily). If a comment has no positive count for any emotion, we assign it to the `neutral` category.

Let the original GoEmotions label set be

$$\mathcal{E}_{28} = \{\texttt{admiration}, \texttt{amusement}, \ldots, \texttt{surprise}, \texttt{neutral}\}.$$

Following the taxonomy in Demszky et al. (2020), we map these 28 fine-grained emotions into four coarse affective groups:

$$\begin{aligned}
\text{Positive} &= \{\texttt{admiration}, \texttt{amusement}, \texttt{approval}, \texttt{caring}, ...\},\\
\text{Negative} &= \{\texttt{anger}, \texttt{annoyance}, \texttt{disappointment}, \texttt{disapproval}, ...\},\\
\text{Ambiguous} &= \{\texttt{confusion}, \texttt{curiosity}, \texttt{realization}, \texttt{surprise}\},\\
\text{Neutral} &= \{\texttt{neutral}\}.
\end{aligned}$$

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

Each comment is first assigned a fine-grained main label in $\mathcal{E}_{28}$, which is then deterministically mapped into one of the four coarse categories. We index these four groups as

$$y_t \in \{0, 1, 2, 3\} \quad \text{corresponding to} \quad \{\text{positive}, \text{negative}, \text{ambiguous}, \text{neutral}\}.$$

Empirically, in our processed corpus we obtain:

- 42.2% positive comments,
- 25.0% negative comments,
- 9.3% ambiguous comments,
- 23.6% neutral comments.

These coarse labels y_t are later used only for evaluation and interpretation of the latent HMM states, not for training.

2.3 Thread-Level Sequence Construction

To capture temporal emotional dynamics, we construct sequences at the Reddit post (thread) level. We group comments by `link_id`, which identifies the original submission, and within each post we sort comments by their `created_utc` timestamps. This yields, for each post, an ordered list of comment IDs

$$\text{seq} = (\text{id}_1, \ldots, \text{id}_T),$$

which we treat as one observation sequence for the HMM. We keep even single-comment threads ($T = 1$) to avoid discarding data.

After aggregation and grouping, our processed dataset contains:

- $44{,}897$ thread-level sequences,
- $58{,}011$ total comments (tokens).

The empirical length distribution is:

- mean length 1.29 comments per thread,
- median length 1 (the 25th, 50th, and 75th percentiles are all 1),
- 90th percentile length 2, 95th percentile 3,
- maximum observed length 25 comments.

Thus most Reddit posts in GoEmotions contain only one annotated comment, but there is a long tail of multi-comment threads that our HMM can use to learn transition dynamics. This length skew is an important limitation of the dataset and will influence how much temporal structure the model can capture.

2.4 Sentence Embedding and Discretization

Hidden Markov Models require discrete observations, whereas comment texts live in a continuous semantic space. We therefore convert each comment into a sentence embedding and then apply vector quantization.

First, we encode each comment using the `bert-base-uncased` transformer. For a given comment, we feed its tokenized text through BERT and extract the final hidden-state of the `[CLS]` token as a fixed-dimensional sentence embedding. To make preprocessing efficient, all BERT embeddings are cached to disk the first time the script is run (in `embeddings.npy` and `embedding_ids.txt`), and subsequent runs reuse this cache without recomputing embeddings.

Second, we run K-Means clustering with $K = 32$ clusters over all sentence embeddings in the dataset. Each comment is assigned to its nearest centroid, producing a discrete observation symbol

$$o_t \in \{0, \ldots, K - 1\},$$

which captures coarse semantic and stylistic similarity between comments. In practice, all 32 clusters are used: the most frequent cluster covers about 5.9% of all comments, and the top ten clusters each account for roughly 2–6% of the data, suggesting a reasonably balanced partition of the sentence embedding space.

2.5 Exported Files for HMM Training

Finally, we export three text files used by the downstream HMM components:

- `observations.txt`: one line per thread, containing the sequence of discrete observation IDs o_t (cluster indices), space-separated.
- `labels.txt`: one line per thread, containing the corresponding sequence of coarse emotion labels $y_t \in \{0, 1, 2, 3\}$, space-separated.
- `map.txt`: one line per thread, containing the original Reddit comment IDs (`id` column) in temporal order, used to map HMM states back to raw comments for qualitative analysis.

These files define the interface between the data processing pipeline and the HMM learning code: the HMM only sees the discrete observation sequences in `observations.txt`, while `labels.txt` and `map.txt` are used for evaluation and interpretation of the learned latent states.

3 Modeling and Inference

Our objective is to model how the underlying emotional state of a Reddit discussion evolves over time. Instead of relying on annotated emotion labels, we adopt an unsupervised approach in which the text of each comment is mapped to a discrete observation symbol, and an HMM is used to infer a latent sequence of high-level emotional states. This formulation allows the model to learn coarse-grained "mood phases" directly from the textual dynamics of a thread.

Observation Construction. Each comment in a Reddit thread is first encoded using the final hidden-layer embedding of `bert-base-uncased`. Since HMMs require discrete observations, we perform vector quantization by applying K-Means clustering with K clusters over all comment embeddings in the dataset. Each comment is therefore mapped to a discrete symbol

$$o_t \in \{1, \ldots, K\},$$

representing its semantic cluster assignment. A Reddit thread of T comments becomes an observation sequence

$$O = (o_1, o_2, \ldots, o_T),$$

where o_t captures coarse semantic and stylistic information of the t-th comment.

Hidden Emotion States. We posit that the discussion at each time step is governed by a latent emotional state that is not directly observed but influences the style and content of comments. Let

$$z_t \in \{1, \ldots, S\}$$

denote the hidden emotion state at time t, where S is a hyperparameter controlling the granularity of the learned emotional phases (e.g., escalation, positivity, resignation). The latent state sequence $Z = (z_1, \ldots, z_T)$ is assumed to evolve according to Markov dynamics.

Hidden Markov Model. We model the joint dynamics of hidden emotional states and observed semantic clusters using a multinomial HMM

$$\lambda = (A, B, \pi),$$

where:

- $A_{ij} = P(z_{t+1} = j \mid z_t = i)$ is the transition matrix governing how emotional states evolve within a thread,
- $B_{ik} = P(o_t = k \mid z_t = i)$ is the emission matrix relating latent emotional states to observable BERT-cluster symbols,
- $\pi_i = P(z_1 = i)$ is the initial distribution over emotional states.

Parameters are estimated with the Baum–Welch algorithm (EM), maximizing the likelihood of all thread-level observation sequences.

Learning: EM Algorithm Given the HMM parameters $\lambda = (A, B, \pi)$ and a collection of Reddit threads, we learn the model in an unsupervised fashion using the Baum–Welch algorithm, i.e., the Expectation–Maximization (EM) algorithm specialized to HMMs.

Multiple-sequence likelihood. Let $\{O^{(n)}\}_{n=1}^{N}$ denote the set of Reddit threads after discretization, where thread n is

$$O^{(n)} = \big(o_1^{(n)}, \ldots, o_{T_n}^{(n)}\big), \quad o_t^{(n)} \in \{1, \ldots, K\}.$$

We assume each thread is generated independently by the same HMM. The training objective is to maximize the total log-likelihoodv

$$\mathcal{L}(\lambda) = \sum_{n=1}^{N} \log p\Big(O^{(n)} \mid \lambda\Big).$$

Forward–backward recursions. For a single sequence $O = (o_1, \ldots, o_T)$, the *forward* variables are

$$\alpha_t(i) = p(o_1, \ldots, o_t, z_t = i \mid \lambda), \quad i \in \{1, \ldots, S\}.$$

They satisfy the recursions

$$\alpha_1(i) = \pi_i \, B_{i,o_1},$$

$$\alpha_{t+1}(j) = \Big[\sum_{i=1}^{S} \alpha_t(i) A_{ij}\Big] B_{j,o_{t+1}}, \quad t = 1, \ldots, T-1.$$

Similarly, the *backward* variables are

$$\beta_t(i) = p(o_{t+1}, \ldots, o_T \mid z_t = i, \lambda),$$

with recursions

$$\beta_T(i) = 1,$$

$$\beta_t(i) = \sum_{j=1}^{S} A_{ij} B_{j,o_{t+1}} \beta_{t+1}(j), \quad t = T-1, \ldots, 1.$$

The log-likelihood of the sequence is

$$p(O \mid \lambda) = \sum_{i=1}^{S} \alpha_T(i).$$

In our implementation, these recursions are carried out in log-space to avoid numerical underflow.

E-step: posterior expectations. Using $\alpha_t(i)$ and $\beta_t(i)$, we compute for each sequence and each time step the posterior probability of being in state i,

$$\gamma_t(i) = p(z_t = i \mid O, \lambda) = \frac{\alpha_t(i)\, \beta_t(i)}{p(O \mid \lambda)}.$$

We also compute the joint posterior over consecutive states,

$$\xi_t(i, j) = p(z_t = i, z_{t+1} = j \mid O, \lambda) = \frac{\alpha_t(i) A_{ij} B_{j,o_{t+1}} \beta_{t+1}(j)}{p(O \mid \lambda)}.$$

For multiple sequences, we repeat this forward–backward computation for each $O^{(n)}$ and aggregate expected counts across all threads:

$$\text{expected initial counts:} \quad \hat{\pi}_i = \sum_{n=1}^{N} \gamma_1^{(n)}(i),$$

$$\text{expected transition counts:} \quad \hat{A}_{ij} = \sum_{n=1}^{N} \sum_{t=1}^{T_n - 1} \xi_t^{(n)}(i, j),$$

$$\text{expected emission counts:} \quad \hat{B}_{ik} = \sum_{n=1}^{N} \sum_{t=1}^{T_n} \gamma_t^{(n)}(i)\, \mathbb{I}\big[o_t^{(n)} = k\big],$$

where $\mathbb{I}[\cdot]$ is the indicator function and superscripts (n) denote the thread index.

M-step: parameter re-estimation. Given these expected counts, the M-step replaces each parameter with the normalized expected frequency of the corresponding event under the current posterior:

$$\pi_i^{\text{new}} = \frac{\hat{\pi}_i}{\sum_{j=1}^{S} \hat{\pi}_j} = \frac{1}{N} \sum_{n=1}^{N} \gamma_1^{(n)}(i),$$

$$A_{ij}^{\text{new}} = \frac{\hat{A}_{ij}}{\sum_{j'=1}^{S} \hat{A}_{ij'}} = \frac{\sum_{n=1}^{N} \sum_{t=1}^{T_n-1} \xi_t^{(n)}(i,j)}{\sum_{n=1}^{N} \sum_{t=1}^{T_n-1} \gamma_t^{(n)}(i)},$$

$$B_{ik}^{\text{new}} = \frac{\hat{B}_{ik}}{\sum_{k'=1}^{K} \hat{B}_{ik'}} = \frac{\sum_{n=1}^{N} \sum_{t=1}^{T_n} \gamma_t^{(n)}(i)\, \mathbb{I}\big[o_t^{(n)} = k\big]}{\sum_{n=1}^{N} \sum_{t=1}^{T_n} \gamma_t^{(n)}(i)}.$$

We iterate E and M-steps for a fixed number of iterations, obtaining the learned parameters (A, B, π) that locally maximize the likelihood of all thread-level observation sequences.

Inference. Given a new discussion thread with observation sequence $O = (o_1, o_2, \ldots, o_T)$, we infer its most likely latent emotional trajectory using the Viterbi algorithm:

$$\hat{Z} = \arg\max_{Z} p(Z, O \mid \lambda),$$

where $\lambda = (\pi, A, B)$ represents our trained HMM parameters and $Z = (z_1, z_2, \ldots, z_T)$ is the hidden state sequence.

Viterbi algorithm implementation. The Viterbi algorithm efficiently computes the most probable state sequence through dynamic programming in log-space to avoid numerical underflow. We define the Viterbi variable:

$$\ell_t(i) = \max_{z_1, \ldots, z_{t-1}} \log p(z_1, \ldots, z_{t-1}, z_t = i, o_1, \ldots, o_t \mid \lambda)$$

The recursion proceeds as:

$$\ell_1(i) = \log \pi_i + \log B_{i,o_1},$$
$$\ell_t(j) = \max_i [\ell_{t-1}(i) + \log A_{ij}] + \log B_{j,o_t}, \quad t = 2, \ldots, T$$

We maintain backpointers $\phi_t(j) = \arg\max_i [\ell_{t-1}(i) + \log A_{ij}]$ to reconstruct the optimal path. The final decoded path is obtained by:

$$z_T^* = \arg\max_i \ell_T(i),$$
$$z_t^* = \phi_{t+1}(z_{t+1}^*), \quad t = T-1, \ldots, 1$$

Numerical stability. To handle numerical underflow in log-space computations, all probability matrices are clipped to the range $[10^{-12}, 1.0]$ before taking logarithms:

$$\log A_{ij} \leftarrow \log(\text{clip}(A_{ij}, 10^{-12}, 1.0)),$$
$$\log B_{ik} \leftarrow \log(\text{clip}(B_{ik}, 10^{-12}, 1.0)),$$
$$\log \pi_i \leftarrow \log(\text{clip}(\pi_i, 10^{-12}, 1.0))$$

Multi-sequence inference. For multiple discussion threads, we apply Viterbi decoding independently to each observation sequence $O^{(n)} = (o_1^{(n)}, \ldots, o_{T_n}^{(n)})$, yielding decoded paths $\hat{Z}^{(n)} = (z_1^{(n)}, \ldots, z_{T_n}^{(n)})$ for $n = 1, \ldots, N$. Each sequence is processed independently, making the algorithm naturally parallelizable for large-scale thread analysis.

Evaluation. To assess the semantic validity of our unsupervised model, we implement a post-hoc evaluation to align the learned latent states with ground-truth emotion labels. We first infer the most likely hidden state sequence $\hat{Z}$ for each thread using the Viterbi algorithm and construct a co-occurrence matrix between these states and the annotated GoEmotions labels. Each latent state s is then mapped to the emotion class l_s^* that maximizes co-occurrence frequency (majority voting). Finally, we quantify the model's performance by calculating the sentence-level accuracy, defined as the proportion of sentences where the mapped state prediction matches the ground-truth emotion label.

4 Results and Discussion

In this experiment, we trained a Hidden Markov Model with $S = 4$ latent states using observation sequences derived from $K = 32$ semantic clusters. The model was trained on 44,897 Reddit threads.

State-to-Label Mapping. To interpret the semantics of the learned latent states, we computed the co-occurrence matrix between the Viterbi-decoded hidden states and the ground-truth emotion labels. As shown in Table 1, there is a significant class imbalance in the dataset.

We assigned each latent state to the emotion label with the highest frequency (highlighted in bold in the table). The mapping results are as follows: (1) State 0 maps to Label 1 (mixed, but highest count). (2) State 1, 2, and 3 all map to Label 0.

Table 1: State-Label Co-occurrence Matrix. The rows represent the unsupervised HMM states, and the columns represent the ground-truth emotion labels. Bold values indicate the dominant label for each state used for mapping.

HMM State	Label 0	Label 1	Label 2	Label 3	Mapped Label
State 0	819	**914**	237	907	Label 1
State 1	**13,595**	7,648	3,489	7,551	Label 0
State 2	**6,839**	3,361	904	2,702	Label 0
State 3	**3,220**	2,549	736	2,540	Label 0

This distribution indicates a "mode collapse" issue, where the majority class (Label 0) dominates the latent space, causing three out of four states to converge to the same label.

Model Accuracy. Based on the mapping established in Table 1, the final sentence-level accuracy of the model is 42.35%. While this performance exceeds the random baseline, the high confusion between states (particularly for Label 2 and Label 3, which are never predicted as the primary class) suggests that the unsupervised HMM struggles to distinguish finer emotional nuances without supervision.

Inference Result. Based on the mapping in Table 2, it presents a representative thread in which the HMM recovers a coherent latent emotional trajectory that aligns with the human-annotated labels. The model correctly identifies two initial positive comments, detects a sharp tonal shift in the third comment—reflected by a transition into a state mapped to the negative class—and subsequently returns to a positive state as the discussion de-escalates. Although multiple latent states map to the same label, their differentiated assignments indicate that the model captures semantic distinctions beyond the four emotion categories. This example shows that, despite global class imbalance, the unsupervised HMM can still model local emotional dynamics and identify meaningful sentiment shifts within short Reddit threads.

Example Sequence 2929

Accuracy: 1.00 Transitions: positive→ negative, negative→ positive

Table 2: Emotion Transition Example Sequence. The rows show consecutive positions in a dialogue. Bold entries in the **Prediction** column indicate correct predictions.

Pos	Ground Truth	Prediction	HMM State	Text
1	positive	**positive**	2	Please don't use it as toothpaste. It will destroy your teeth.
2	positive	**positive**	2	Yesss to all the frosting.
3	negative	**negative**	0	Why are you so fxxking concerned with what someone else uses to clean their teeth? You sound like...
4	positive	**positive**	3	Now that's a beautiful cake but that's way way WAAAAAAY too much icing for me... Added a good 5 i...

5 Conclusion

In this work, we presented a probabilistic framework for modeling the evolution of emotions in Reddit threads using a sentence-level Hidden Markov Model. By quantizing BERT embeddings into discrete observations, we enabled the HMM to learn latent emotional transitions without relying on direct supervision during the training phase.Our experimental results demonstrate that while the HMM can capture some underlying structure in the conversation dynamics, it is heavily influenced by the class imbalance inherent in the GoEmotions dataset. The model achieved a sentence-level alignment accuracy of 42.35%, with the majority of latent states mapping to the most frequent emotion class. This suggests that while the textual dynamics (transitions between semantic clusters) are learnable, the unsupervised states do not perfectly align with human-annotated emotion categories without additional constraints.

6 Reflections & Contributions

6.1 Suggestions for Future Students

For future iterations of this project, we recommend starting early with data preprocessing and embedding caching, as these steps are both time-consuming and easy to underestimate. When designing the HMM, it is helpful to experiment with different numbers of hidden states and cluster sizes K rather than fixing them a priori, and to monitor not only accuracy but also state–label co-occurrence to detect mode collapse. Finally, we suggest reserving time for qualitative analysis of a few representative threads, since inspecting concrete sequences is often more informative than global metrics when debugging unsupervised models.

6.2 Individual Contributions

Weizhi Li

- Contribution: Project idea discussion; Data source finding and data processing (including code); Report writing on data sourcing and processing.
- Reflection: From this project, I gained hands-on experience building an end-to-end pipeline from raw Reddit data to an HMM-based emotional dynamics model, and I better understood the practical challenges of class imbalance, embedding discretization, and interpreting unsupervised latent states in real-world text.

Yuxuan Liu

- Contribution: Project idea discussion; Code for model learning (EM algorithm on HMM); Report writing, mainly on the modeling and learning algorithm.
- Reflection: In applying the EM algorithm, I learned how to translate lecture concepts into real-world scenarios. Because our project involves multiple sequences, the M-step differs from the single-sequence version discussed in class. I had to extend the equations to handle multi-sequence data by running forward–backward on each sequence and aggregating the results.

Hanrui Fan

- Contribution: Project idea discussion; Formula for Viterbi Algorithm; Code for vectorized Viterbi Inference, and implement reverse mapping to find suitable inference result to discuss; Analysis on Inference Result.
- Reflection: From this project, I had deep understanding of the application of unsupervised probabilistic models on real-world text data. It provides a valuable experience in reasoning, designing, implementing and evaluating learning systems from scratch. Working through Viterbi decoding, I have deepened my understanding of probabilistic inference and how to reconstruct and interpret from discrete textual observations. This project also strengthen my skills of communication, planning, collaboration, as our team worked together to refine our approach iteratively and identify solutions as our designed problem constraints.

Jinzhou Tang

- Contribution: Project idea discussion; Design for Evaluation Protocols; Code for evaluation (Model Accuracy and State-label mapping); Report writing, mainly Evaluation & Results and Discussion parts.
- Reflection: From this project, I learned the complexities of defining and measuring success for unsupervised tasks where ground-truth labels are not used during training. Designing the evaluation protocol taught me how to bridge the gap between statistical latent states and human-interpretable categories using post-hoc mapping. The process of diagnosing the "mode collapse" in our results gave me practical insight into how data skewness affects the EM algorithm, teaching me that rigorous data balancing and thoughtful metric design are just as critical as the modeling algorithm itself.

6.3 Note on GenAI

We used GenAI to brainstorm project ideas, search for relevant datasets, get coding hints, and improve the writing of our report.

References

Demszky Dorottya, Movshovitz-Attias Dana, Ko Jeongwoo, Cowen Alan, Nemade Gaurav, Ravi Sujith. GoEmotions: A Dataset of Fine-Grained Emotions // 58th Annual Meeting of the Association for Computational Linguistics (ACL). 2020.

A Appendix

Example GoEmotions Data Entry (Metadata Only)

text	id	author	subreddit	link_id	parent_id	created_utc
That game hurt.	eew5j0j	Brdd9	nrl	t3_ajis4z	t1_eew18eq	1548381039.0

Table 3: Example metadata fields for a GoEmotions entry.

Emotion Annotations for the Example Entry

adm.	amus.	anger	annoy.	appr.	caring	conf.	curios.	desire	disapp.	disappr.	disgust	embarr.	excite
0	0	0	0	0	0	0	0	0	0	0	0	0	0

fear	grat.	grief	joy	love	nerv.	optim.	pride	realiz.	relief	remorse	sadness	surprise	neutral
0	0	0	0	0	0	0	0	1	0	0	0	0	0

Table 4: Emotion labels for the same GoEmotions entry (all 28 categories).